CSCI 5540 – Fall 26

Tutorial 6

DENG Yangtao

Credits to CMU Machine Learning Systems

What we will cover

1 **Swizzle**

Map a logical element to a bank-friendly physical address.

2 **Tensor Cores**

Accumulate matrix tiles while reusing data.

3 **TMA and tiled GEMM**

Move the right layout, wait for readiness, then compute.

Main question: how do we keep the matrix-multiply hardware supplied with data?

Three jobs along the GEMM data path

$$C_{M\times N} = A_{M\times K} B_{K\times N}$$

Global memory	Shared memory	Accumulators
Full A and B	Reusable A / B tiles	Partial C tile
TMA: move tiles	Swizzle: place tiles	MMA: consume tiles

Reuse, conflict avoidance, and asynchronous transfer solve different bottlenecks.

A bank conflict serializes distinct addresses

Toy model: 4 banks, 1 word per request; bank = word address mod 4

Pattern	Thread 0	Thread 1	Thread 2	Thread 3
Consecutive addresses	0	1	2	3
Target bank	0	1	2	3
Strided addresses	0	4	8	12
Target bank	0	0	0	0

Different addresses in one bank → conflict. Same-address reads may broadcast.

XOR swizzle spreads a logical column across banks

Physical column = logical column XOR row

Row-major

Bank	0	1	2	3	4	5	6	7
0	**0**	1	2	3	4	5	6	7
1	**0**	1	2	3	4	5	6	7
2	**0**	1	2	3	4	5	6	7
3	**0**	1	2	3	4	5	6	7
4	**0**	1	2	3	4	5	6	7
5	**0**	1	2	3	4	5	6	7
6	**0**	1	2	3	4	5	6	7
7	**0**	1	2	3	4	5	6	7

XOR-swizzled

Bank	0	1	2	3	4	5	6	7
0	**0**	1	2	3	4	5	6	7
1	1	**0**	3	2	5	4	7	6
2	2	3	**0**	1	6	7	4	5
3	3	2	1	**0**	7	6	5	4
4	4	5	6	7	**0**	1	2	3
5	5	4	7	6	1	**0**	3	2
6	6	7	4	5	2	3	**0**	1
7	7	6	5	4	3	2	1	**0**

Highlighted logical column 0: one bank before → eight banks after.

SWIZZLE_128B operates on 16-byte sectors

32 banks × 4 bytes = 128 bytes per row

Sector 0	Sector 1	Sector 2	Sector 3	Sector 4	Sector 5	Sector 6	Sector 7
0–3	4–7	8–11	12–15	16–19	20–23	24–27	28–31

Bank IDs

1 sector = 16 B = 8 FP16 = 4 FP32

One 128B atom: 8 rows × 128 B = 1024 B

Sector index changes; the offset of a value inside its sector stays the same.

Worked example: 32 reads, 32 distinct banks

Row	Logical sector	Banks without swizzle	Banks with swizzle
0	0	0–3	0–3
1	0	0–3	4–7
2	0	0–3	8–11
3	0	0–3	12–15
4	0	0–3	16–19
5	0	0–3	20–23
6	0	0–3	24–27
7	0	0–3	28–31

Same 128 bytes requested: 8-way conflicts → conflict-free bank assignment.

Follow one value to its physical byte address

r = 3 logical sector q = 1 FP32 word t = 2

Step	Calculation	Result
Within-sector byte offset	4 × t	8 B
Physical sector	1 XOR 3 (001 XOR 011)	2
Physical byte offset	3 × 128 + 2 × 16 + 8	424 B
Physical bank	(424 / 4) mod 32	10

Row-major offset = 408 B. Swizzled offset = 424 B. Logical value is unchanged.

Choose a swizzle width that fits the tile

Source layout width	16 B sectors / row	FP16 values / row
128 B	8	64
64 B	4	32
32 B	2	16
16 B baseline	1	8

Prefer wide contiguous segments when supported; validate the full layout and access pattern.

Tiling: a wide matrix becomes groups of narrow rows

Logical tile: 16 × 128 FP16

Group 0: columns 0–63
16 × 64 FP16 = 2048 B

Group 1: columns 64–127
16 × 64 FP16 = 2048 B

$$g = \lfloor c/64 \rfloor, \quad u = c \bmod 64, \quad \text{offset}_{elem} = g \cdot 1024 + r \cdot 64 + u$$

(r, c) = (3, 72) → (g, r, u) = (1, 3, 8) → 2448 B

First choose the grouped tile layout. Then apply the sector swizzle within its atoms.

Tensor Cores accumulate matrix tiles

$$C_{16\times16} \mathrel{+}= A_{16\times16}\, B_{16\times16}$$

K slice	Matrix update	All-ones result
0–15	$C \leftarrow 0 + A_0B_0$	Every C element = 16
16–31	$C \leftarrow C + A_1B_1$	Every C element = 32

Keep C live across K slices; reset it only for a new output tile.

A logical tile operation may require multiple hardware MMA instructions.

The compute tile must agree with the memory layout

16 × 16 FP16 operand = 512 B

8 × 8 FP16 128 B	**8 × 8 FP16 128 B**
8 × 8 FP16 128 B	**8 × 8 FP16 128 B**

4 subtiles × 128 B = 512 B

Row and column reads both matter

Transpose choices change the access pattern.

A compute subtile and a swizzle atom describe different units.

TMA moves a tensor tile using a descriptor

Descriptor

Base pointer, shape, byte strides
Tile shape, dtype, swizzle

Thread issues copy → **TMA transfers tile** → **Consumer waits** → **MMA reads tile**

Asynchronous launch returns before the tile is ready.

The descriptor defines the transfer; completion synchronization makes it safe to consume.

2D TMA example: copy an 8 × 64 FP16 tile

Field / quantity	Value	Units / meaning
Global tensor	16 × 128	FP16 values; row-major
globalDim	[128, 16]	Elements; contiguous axis first
globalStrides	[256]	Bytes between rows
boxDim	[64, 8]	Elements; 1024 B total
Copy coordinates	[64, 8]	Start column, start row

Source start = 8 × 256 + 64 × 2 = 2176 B. Destination uses its own tile layout.

From a 2D matrix to a 3D tile view

$$(r, c) \longrightarrow (g, r, u), \qquad g = \lfloor c/64 \rfloor, \quad u = c \bmod 64$$

Concrete copy from row-major global memory	Destination in shared memory
Rows 0–15, columns 0–63	Group 0: 16 × 64 FP16
Rows 0–15, columns 64–127	Group 1: 16 × 64 FP16

3D describes the tile coordinates; physical strides still constrain the transfer.

A reshape describes indexing. TMA must actually produce the intended destination layout.

Two different completions protect a shared buffer



Wait for copy completion

Wait for consumer completion

Read too early → incomplete input

Overwrite too early → corrupted input

Ready to read and safe to overwrite are different conditions.

Running GEMM: 256 × 256, one 128 × 128 output tile

Dimension / tile	Size	Role
Full problem	M = N = K = 256	C = A × B
Output tile	BM = BN = 128	One logical block computes it
Staged K chunk	BK = 128	2 outer chunks
Logical compute K slice	16	8 inner slices per chunk

One output tile: 2 outer chunks × 8 inner slices = 16 logical K steps.

Trace both K loops for the first output tile

Outer	Inner	Global K range	What happens
0	0	0–15	Start accumulating into C
0	7	112–127	Finish first staged chunk
1	0	128–143	Keep C; use new A / B tiles
1	7	240–255	Finish reduction; write C

At each outer step: load A[:, Kchunk] and B[Kchunk, :] for this output tile.

Global K start = 128 × outer + 16 × inner. The accumulator survives both loops.

Tile reuse buys arithmetic intensity, but costs memory

Resource / work	Calculation	Result
One FP16 input tile	128 × 128 × 2 B	32 KiB
A + B, one buffer pair	2 × 32 KiB	64 KiB
A + B, two buffer pairs	2 × 64 KiB	128 KiB
Work per staged K chunk	2 × 128 × 128 × 128	4.19 MFLOP

Input-only arithmetic intensity: 4,194,304 FLOPs / 65,536 B = 64 FLOPs/B.

Double buffering overlaps copying with computation

Time →

TMA	**Load K0 buffer A**	**Load K1 buffer B**
Compute	**Compute K0 buffer A**	**Compute K1 buffer B**

Overlap

Keep the C accumulator live across both chunks.

Each named buffer contains both A and B input tiles. Reuse only after consumers finish.

Design the layout from the consumer backward

1 **Choose the compute and shared-memory tiles**

Check operand layout, reuse, and resource usage.

2 **Choose swizzle and tile grouping**

Keep the producer and consumer address mappings consistent.

3 **Configure copies, synchronization, and buffering**

Then measure full-kernel throughput and correctness.

Quick check: reason about addresses and lifetimes

1. r = 3, logical sector = 1. What is the physical sector after XOR?
2. How many bytes are in an 8 × 64 FP16 transfer box?
3. K = 256, BK = 128, inner K = 16. How many chunks and steps?
4. TMA has completed. Is its shared-memory buffer safe to overwrite?

Use three checks: byte units, logical-to-physical mapping, and buffer lifetime.

Three ideas to remember

1 **Swizzle**

Spread accesses across banks without changing the logical matrix.

2 **Tensor Cores**

Reuse operand tiles and keep accumulating across K.

3 **TMA**

Move tiles asynchronously; wait before reading and before reusing buffers.

Thanks!